iGloo: A CLI-Mediated Architecture for Operating-System Installation and Migration

Technical Architecture White Paper and Interface Baseline

Gilles D'huyvetter[1]
[1]iGloo Project

Version 1.0 - Final Architecture Baseline - 6 October 2026

Abstract

Operating-system installation and migration are high-impact endpoint operations. They modify storage, boot configuration, installer state, user data, credentials, and hardware-specific configuration, often across a reboot and across two operating systems. When a graphical client, iGloo One, an enterprise fleet controller, and third-party tooling each obtain separate access to that behavior, execution semantics drift and safety controls become difficult to reason about.

This paper specifies the iGloo architecture around one deliberately narrow rule: *iGloo Core* is the execution engine, and *iGloo CLI* is the single canonical executable gateway to that engine. *iGloo Community*, *iGloo One*, the *iGloo Fleet* endpoint Agent, human operators, scripts, and independent third-party frontends all consume endpoint capabilities through the CLI. The CLI exposes two interaction modes over the same capability implementation: an ergonomic human interface and a strict machine protocol based on versioned, typed command semantics. A small reusable client library may manage process invocation for first-party applications, but it does not contain migration behavior.

The architecture further separates consumer-side extensibility from provider-side extensibility. Above the CLI, users, One, enterprise orchestration, OEMs, managed-service providers, and Linux-distribution maintainers can build independent interaction models. Below Core, distribution providers supply target-specific compatibility, installer configuration, boot semantics, and first-boot behavior without leaking those details into clients. Long-running state, cross-reboot continuation, resource locking, privilege, recovery, and verification remain Core responsibilities rather than properties of the invoking UI or CLI process.

The result is a systems architecture intended to make iGloo sustainable as both a product family and an extensible installation/migration platform: one execution authority, one executable gateway, one machine protocol, multiple clients, and independently replaceable distribution providers. This document is an architecture baseline and interface specification, not a standards-track publication or a claim that every target capability is already implemented.

Keywords: operating-system migration; installation automation; command-line interface; machine protocol; systems architecture; Linux; fleet orchestration; iGloo One; interoperability; recovery; endpoint management.

Contents

1 Scope and design goals **5**
1.1 Scope . . . 5
1.2 Primary goals . . . 5
1.3 Non-goals . . . 5

2 Architectural thesis **6**
2.1 The problem with multiple execution authorities . . . 6
2.2 Canonical logical architecture . . . 6
2.3 Consumer and provider extension axes . . . 6

3 Architectural invariants **6**

4 Component responsibilities **8**
4.1 iGloo Core . . . 8
4.2 iGloo CLI . . . 8
4.3 iGloo Command Contract . . . 9
4.4 iGloo CLI Client . . . 9
4.5 iGloo Community . . . 9
4.6 iGloo One . . . 9
4.7 iGloo Fleet . . . 9
4.8 Third-party clients . . . 10
4.9 Distribution providers . . . 10

5 CLI interaction model **10**
5.1 One capability implementation, two renderers . . . 10
5.2 Human mode . . . 10
5.3 Machine mode . . . 10
5.4 stdout, stderr, and event framing . . . 11
5.5 Behavioral flags . . . 11

6 The iGloo Command Contract **11**
6.1 Capability descriptor . . . 11
6.2 Capability taxonomy . . . 12
6.3 Request and result envelopes . . . 13
6.4 Error taxonomy . . . 13
6.5 Versioning . . . 13
6.6 Canonicalization and evidence hashes . . . 14

7 Core execution model **14**
7.1 State transition model . . . 14
7.2 Preconditions and postconditions . . . 14
7.3 Stale-state and TOCTOU protection . . . 14
7.4 Intent, artifact, target, allocation, and operation identities . . . 15
7.5 Risk and authority . . . 15
7.6 Privilege separation . . . 16

8 Durable operations and cross-reboot continuation **16**
8.1 Core owns operation state . . . 16
8.2 Lifecycle . . . 16
8.3 Progress and reconnection . . . 17
8.4 Cancellation . . . 17

8.5 Idempotency and retry . . . 17

9 Concurrency and resource ownership 17

10 Secrets and sensitive data 17
10.1 No secrets in command-line arguments . . . 17
10.2 Logging and results . . . 18
10.3 Fleet privacy boundary . . . 18

11 Distribution-provider architecture 18
11.1 Provider role . . . 18
11.2 Provider constraints . . . 18

12 Client integration 18
12.1 iGloo Community . . . 18
12.2 iGloo One . . . 19
12.3 iGloo Fleet . . . 19
12.4 Third-party and distro-owned frontends . . . 19

13 Security and trust boundaries 20
13.1 Threat model . . . 20
13.2 CLI location and substitution resistance . . . 20
13.3 Approval artifacts . . . 20

14 Recovery and failure containment 21
14.1 Recovery vocabulary . . . 21
14.2 Verification . . . 21
14.3 CLI process failure versus operation failure . . . 21

15 Observability and evidence 21

16 Conformance and compatibility 22
16.1 Client conformance . . . 22
16.2 Provider conformance . . . 22
16.3 First-party semantic parity . . . 22
16.4 Architecture tests . . . 22

17 Reference interaction flows 23
17.1 Community inspection . . . 23
17.2 One tool use . . . 23
17.3 Fleet dry-run . . . 23
17.4 Distribution-owned frontend . . . 23
17.5 Cross-reboot migration . . . 24

18 Implementation status and migration plan (non-normative) 24
18.1 Status at publication . . . 24
18.2 Remaining convergence sequence . . . 24

19 Validation strategy 25
19.1 Protocol tests . . . 25
19.2 Core safety tests . . . 26
19.3 Cross-client parity . . . 26
19.4 Provider matrix . . . 26

20 Open engineering questions **26**

21 Conclusion **27**

1 Scope and design goals

1.1 Scope

The paper addresses the software architecture by which iGloo exposes endpoint installation, migration, inspection, verification, and recovery capabilities through multiple clients without duplicating execution logic. It covers:

- the executable boundary between clients and the execution engine;
- the role of the *iGloo Command Contract* and the machine-readable CLI protocol;
- the separation between human CLI rendering and machine invocation;
- first-party clients: Community, One, and the Fleet endpoint Agent;
- third-party frontends, scripts, OEM/MSP tooling, and distribution-owned migrators;
- distribution-provider extensibility below Core;
- long-running and cross-reboot operation state;
- concurrency, stale-state protection, privilege, secrets, recovery, and verification;
- versioning, conformance, and architecture regression barriers.

The paper is deliberately not a business, funding, marketing, or digital-sovereignty document. Those topics can motivate deployment, but they do not define the execution boundary.

1.2 Primary goals

The architecture is designed to satisfy six engineering goals:

1. **One execution authority.** Endpoint-domain behavior is implemented once in Core.
2. **One executable gateway.** All first-party endpoint clients reach Core through the CLI process boundary.
3. **One machine protocol.** Automation, first-party products, and third parties use versioned structured semantics rather than parsing human terminal text.
4. **Human usability without semantic forks.** Human CLI commands and machine requests reach the same capability handlers.
5. **Independent client evolution.** Community, One, Fleet, and third-party clients can change without understanding internal engine composition.
6. **Independent provider evolution.** Distribution-specific installer behavior remains below Core and does not leak into every client.

1.3 Non-goals

This architecture does not imply that:

- every Linux distribution or Windows workload can be migrated safely;
- every operation is reversible;
- One is required for any iGloo operation;
- Fleet must be cloud-hosted;
- the CLI process must remain alive across reboot;
- third-party clients or providers are trusted merely because they use the iGloo protocol;

- every internal Core function is public;
- a prepared plan authorizes execution;
- a zero process exit code is sufficient proof of successful migration;
- the current implementation already satisfies every requirement in this document.

2 Architectural thesis

2.1 The problem with multiple execution authorities

A conventional migration application can place most behavior behind one graphical workflow. That model stops scaling when the same engine must also serve a command line, One, enterprise orchestration, scripts, and distribution-owned tooling. If each client invokes internal services directly or reimplements sequencing, the repository gradually acquires multiple authorities for the same high-impact operation.

Let E denote the authoritative semantics of an endpoint capability and C_i a client-specific execution path. A drift-prone design tends toward

$$E_1 \neq E_2 \neq \cdots \neq E_n, \tag{1}$$

where behavior diverges as each client accumulates its own validation, compatibility rules, retries, and failure handling.

The target iGloo architecture instead requires

$$\forall c \in C, \qquad c \rightarrow \text{CLI} \rightarrow \text{Core}, \tag{2}$$

where C contains Community, One, Fleet Agent, humans, scripts, and third-party clients. Core remains the single owner of endpoint-domain execution semantics.

2.2 Canonical logical architecture

The diagram is intentionally simple. Internal Core implementation may remain decomposed into projects for preflight, ISO acquisition, migration, recovery, USB writing, runtime dispatch, provider discovery, and other concerns. Those details are not client dependencies.

2.3 Consumer and provider extension axes

The architecture has two independent extension axes:

- **Consumer-side extension**: a new client reuses existing capabilities by speaking to CLI. It does not need to know which internal assembly performs an operation.
- **Provider-side extension**: a new Linux distribution supplies installer-specific behavior below Core. Existing clients do not need distribution-specific branches.

This symmetry is central to the platform. A distribution maintainer can own both a provider plug-in and a completely independent frontend while still reusing the same engine.

3 Architectural invariants

The following invariants are normative. Violating them is an architecture change, not an implementation detail.

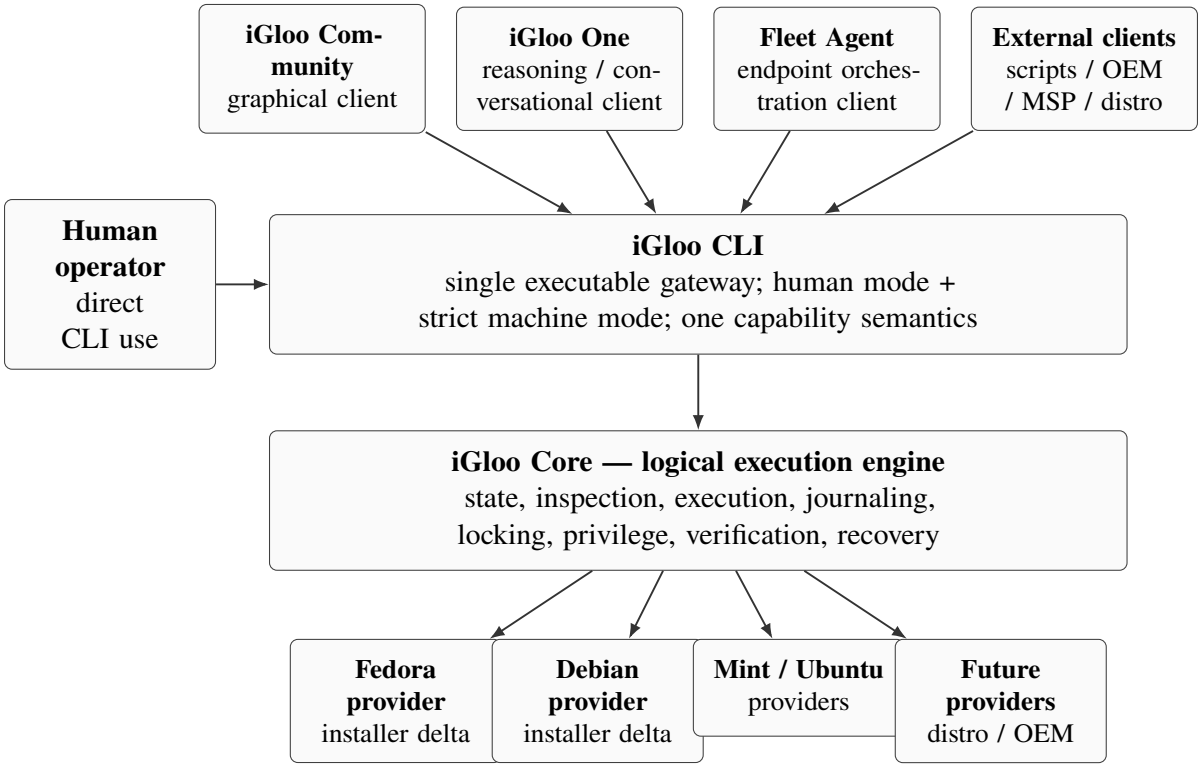


Figure 1: Canonical logical architecture. All endpoint-facing consumers converge on iGloo CLI. CLI is the sole first-party executable gateway into Core. Distribution providers extend the target side below Core.

Table 1: Normative architecture invariants.

ID	Invariant	Requirement
A1	Single execution authority	All endpoint-domain mutation MUST be implemented by Core. Clients may request or present operations, but MUST NOT independently implement disk, boot, migration, installer, or recovery mutation semantics.
A2	Single executable gateway	Community, One, Fleet Agent, and first-party automation MUST invoke endpoint capabilities through iGloo CLI rather than Core.Host or execution assemblies directly.
A3	One semantic path	Human CLI syntax and machine-mode requests MUST resolve to the same capability implementation.
A4	Machine protocol purity	Machine-mode stdout MUST contain protocol data only; clients MUST NOT scrape human terminal output.
A5	Core owns durability	Durable operation state, cross-reboot continuation, resource ownership, and recovery obligations MUST be owned below the CLI process boundary.
A6	Fleet Server isolation	Fleet Server MUST NOT invoke CLI or reference endpoint execution assemblies. Only the endpoint Agent may invoke the local CLI.
A7	Provider independence	Distribution providers MUST NOT reference Community, One, Fleet, or CLI. Provider behavior remains below Core.
A8	No authority by intent	A GUI action, One request, Fleet job, or shell command is a request, not authorization. Core MUST independently evaluate required authority and state.

ID	Invariant	Requirement
A9	Observable completion	A mutation MUST NOT be considered successful only because a process exited successfully; capability postconditions SHOULD be verified.
A10	No secrets in argv	Secrets MUST NOT be transported in CLI command-line arguments.

4 Component responsibilities

4.1 iGloo Core

iGloo Core is the logical execution engine. Its boundary is broader than any one physical assembly. Core owns or coordinates the implementation of endpoint-domain operations, including:

- system and hardware inspection;
- target/distro compatibility evaluation;
- ISO acquisition and cryptographic verification;
- migration planning and preparation;
- data staging and installer artifact generation;
- storage and partition operations;
- boot and UEFI operations;
- USB fallback preparation;
- Linux-side continuation and first-boot work;
- operation journaling and cross-reboot state;
- concurrency/resource ownership;
- verification and recovery-facing operations;
- provider discovery and provider-specific installer semantics.

Core is not responsible for WPF presentation, One reasoning/conversation, terminal formatting, Fleet transport, or business policy presentation.

4.2 iGloo CLI

iGloo CLI is a standalone product and the sole canonical executable gateway into Core. It has two externally visible faces:

1. **Human mode**: discoverable command grammar, help, readable tables, prompts where safe, and terminal-oriented progress.
2. **Machine mode**: strict versioned request/event/result protocol intended for Community, One, Fleet Agent, scripts, and independent tooling.

CLI is an adapter and gateway, not a second execution engine. It owns parsing, protocol framing, human rendering, machine rendering, process-level exit mapping, and trusted Core composition. It MUST NOT duplicate the implementation of endpoint capabilities.

Executable identity. The existing Community/WPF application identity `iGloo.exe` is not the CLI. The CLI is the distinct application produced by the `Igloo.Cli` project. Packaging MUST preserve this distinction and provide first-party clients a deterministic way to locate the compatible CLI binary.

4.3 iGloo Command Contract

The *iGloo Command Contract* defines the machine-readable semantics exposed by CLI: capability identifiers, request schemas, result/event schemas, errors, lifecycle rules, version compatibility, risk/authority metadata, and evidence semantics. It is a project specification designed to support independent clients. It is not claimed to be an industry standard.

4.4 iGloo CLI Client

First-party graphical/agent applications should not each reimplement process invocation. A small *iGloo CLI Client* library may provide:

- trusted CLI binary resolution;
- process creation;
- stdin request transport;
- stdout event/result parsing;
- stderr diagnostic capture;
- timeout and cancellation plumbing;
- protocol/version negotiation;
- typed serialization against Command Contract models.

The library MUST NOT reference Core, Runtime, Preflight, ISO, Migration, UsbWriter, or Fleet policy. It is a transport helper, not an execution layer.

4.5 iGloo Community

iGloo Community is the official graphical client. It owns views, navigation, local interaction, confirmations, graphical progress, and result presentation. It does not own endpoint execution semantics. Its canonical backend path is:

$$\text{Community} \rightarrow \text{CLI Client} \rightarrow \text{CLI} \rightarrow \text{Core}. \tag{3}$$

4.6 iGloo One

iGloo One is the reasoning and conversational client. The product name is *One*; it is not a second execution engine and it is not a privileged shell. One may use replaceable language-model or other reasoning backends internally, but those models are implementation choices behind the product boundary. One interprets user intent, selects published iGloo capabilities, explains plans and structured results, and handles uncertainty. Its canonical path is:

$$\text{One} \rightarrow \text{tool adapter} \rightarrow \text{CLI Client} \rightarrow \text{CLI} \rightarrow \text{Core}. \tag{4}$$

4.7 iGloo Fleet

Fleet separates control-plane orchestration from endpoint execution. Fleet Server manages identity, policy, work assignment, approvals, scheduling, and evidence aggregation. The endpoint Agent translates an authorized Fleet work item into a local CLI request. The canonical execution path is:

$$\text{Fleet Server} \rightarrow \text{Fleet Agent} \rightarrow \text{CLI Client} \rightarrow \text{CLI} \rightarrow \text{Core}. \tag{5}$$

Fleet Server never becomes a remote Core and never receives a private local migration manifest merely because Fleet orchestrates the operation.

4.8 Third-party clients

Independent frontends, scripts, OEM tooling, managed-service providers, and distribution-owned migration assistants use the CLI machine interface directly. They do not need Community, One, or Fleet, and they do not need to link against iGloo internal execution assemblies.

4.9 Distribution providers

Providers extend the target side below Core. They may supply compatibility rules, metadata, installer configuration rendering, boot specifications, first-boot payloads, and distribution-specific verification hooks. They do not own general disk/firmware mutation authority.

5 CLI interaction model

5.1 One capability implementation, two renderers

Human and machine modes MUST converge before Core invocation. The intended flow is:

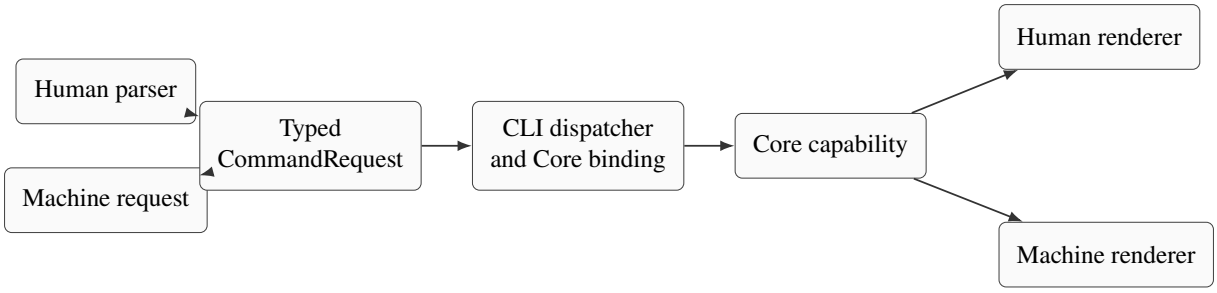


Figure 2: Human and machine interfaces share one typed request path and one Core capability implementation. Presentation diverges only after the semantic result exists.

A design with separate human and machine business handlers is non-conformant because it recreates the execution drift the architecture is intended to remove.

5.2 Human mode

Human mode may expose commands such as:

```
<cli> system inspect
<cli> distros list
<cli> distros compatibility debian
<cli> migration plan --distro debian
<cli> operations status <operation-id>
```

The exact grammar may evolve. Human formatting is not the compatibility boundary; capability semantics are.

5.3 Machine mode

Machine clients MUST NOT construct human command strings and scrape text. CLI provides one strict machine entry point. The exact executable name is packaging-defined; conceptually:

```
<cli> invoke --machine
```

The request is written to stdin. An illustrative request envelope is:

```
{
  "protocolVersion": "1.0",
  "requestId": "8b0f...",
```

```
  "correlationId": "5a12...",
  "capability": "migration.plan",
  "capabilityVersion": "1",
  "input": {
    "providerId": "debian",
    "mode": "dualBoot"
  }
}
```

The example is descriptive; exact V1 JSON Schema belongs to the Command Contract specification. JSON syntax follows RFC 8259 and schema validation should use an explicit machine-readable schema such as JSON Schema [1, 2].

5.4 stdout, stderr, and event framing

In machine mode:

- stdout MUST contain protocol frames only;
- stderr MAY contain local diagnostics but MUST NOT be required to determine semantic success;
- human banners, progress bars, and decorative output MUST NOT contaminate stdout;
- the terminal outcome MUST be unambiguous.

Long-running work should use a structured stream such as JSON Lines. An illustrative sequence is:

```
{"type":"accepted","operationId":"op-7f..."}
{"type":"progress","stage":"download","completed":0.42}
{"type":"progress","stage":"verify","completed":1.0}
{"type":"state","state":"Prepared"}
{"type":"result","status":"success","operationId":"op-7f..."}
```

5.5 Behavioral flags

Generic behavior flags may include `-machine`, `-json`, `-non-interactive`, `-dry-run`, request/correlation identifiers, and timeouts. Product-specific semantic modes such as `-community-mode` or `-fleet-mode` SHOULD NOT exist. Client identity may be supplied as audit context, but must not silently alter capability meaning.

6 The iGloo Command Contract

6.1 Capability descriptor

Every public capability should expose metadata sufficient for discovery and safe invocation. A descriptor should contain at least:

- stable capability identifier and capability version;
- request and result schema identifiers/versions;
- query/plan/mutation classification;
- declared risk class;
- required privilege and authority;
- dry-run/preview availability where meaningful;
- cancellation characteristics;
- long-running/operation semantics;

- relevant provider/environment constraints;
- resource-lock declarations where known.

Capability discovery allows clients to ask what the installed engine can actually do rather than assume support from product version alone.

6.2 Capability taxonomy

A useful initial taxonomy separates observation, planning, mutation, and operation control. Illustrative capability families include:

Table 2: Illustrative capability families. Exact names remain governed by the V1 contract specification.

Capability	Class	Purpose
`system.inspect`	Query	Observe canonical endpoint facts needed for planning.
`system.inspect.detail`	Query	Return a richer public inspection view without leaking private engine objects.
`distro.list` / `distro.inspect`	Query	Discover target providers and public metadata.
`distro.compatibility`	Query	Evaluate provider compatibility against canonical observations.
`artifact.acquire` / `artifact.status`	Low-risk mutation / query	Acquire and revalidate a cryptographically verified provider-scoped artifact without exposing private cache paths.
`migration.inspect`	Query	Observe migration-relevant state and previous iGloo artifacts.
`migration.plan`	Plan	Produce a non-executable reviewable plan and durable PlanId bound to observed state/provider revisions.
`migration.plan.read`	Query	Reopen a saved plan through another CLI process.
`migration.target.bind`	Plan/binding	Bind a saved plan to an opaque stable TargetId without authorizing mutation.
`migration.prepare`	Preparation	Future capability: bind exact allocation, secrets, recovery and approval before creating an executable OperationId.
`migration.execute`	Mutation	Future capability: commit an authorized prepared transition subject to state/authority/recovery gates.
`migration.verify`	Verification	Future capability: evaluate declared postconditions and evidence.
`recovery.inspect`	Query	Assess available recovery evidence/readiness without fabricating restoration guarantees.
`operation.status` / `migration.status`	Control	Reconnect to durable work where an operation journal entry exists.
`operation.cancel`	Control	Request cancellation at defined safe boundaries where implemented.

Not every capability in this table is necessarily implemented at publication. The taxonomy defines direction, not release completeness.

6.3 Request and result envelopes

Requests MUST carry enough identity to support deterministic validation, correlation, replay handling, and version negotiation. Results MUST carry typed status and error semantics rather than force consumers to parse natural-language messages.

An illustrative result envelope is:

```
{
  "protocolVersion": "1.0",
  "requestId": "8b0f...",
  "operationId": "op-7f...",
  "status": "failed",
  "error": {
    "code": "state-changed",
    "retryable": true,
    "details": {
      "requiredAction": "replan"
    }
  }
}
```

6.4 Error taxonomy

A stable top-level error taxonomy should distinguish at minimum:

Table 3: Proposed top-level error classes.

Class	Meaning
`invalid-request`	Syntax, schema, enum, or cross-field validation failed.
`unsupported`	Capability, provider, protocol, version, or environment is unsupported.
`precondition-failed`	Required machine state is not satisfied.
`state-changed`	A previously observed/approved state is stale for the requested operation.
`authority-required`	Additional approval, identity, or privilege is required.
`policy-denied`	Explicit policy refuses the requested operation.
`resource-busy`	A conflicting operation owns a required resource.
`external-dependency`	Network, mirror, package, firmware, or another external dependency failed.
`execution-failed`	Core began an operation but a defined execution step failed.
`verification-failed`	Mutation completed but required postconditions were not demonstrated.
`recovery-required`	Execution stopped in a state requiring explicit recovery handling.
`cancelled`	Cancellation completed at a defined boundary.
`transport-failed`	CLI process/channel failed; operation state may need reconnection.
`internal-error`	Unexpected implementation defect; callers must not infer domain semantics.

Raw exception class names, stack traces, credentials, and arbitrary local paths are not public protocol semantics.

6.5 Versioning

The architecture separates four versions:

- CLI product version;
- command protocol version;
- individual capability version;
- serialized schema version.

They MUST NOT be assumed equal. Semantic Versioning provides useful terminology for compatibility policy, but iGloo must define its own precise breaking-change rules for safety-sensitive commands [3]. A minor compatibility revision must not silently broaden destructive behavior.

6.6 Canonicalization and evidence hashes

Where approvals, plans, or evidence are hash-bound, hashing arbitrary JSON serialization is unsafe because member ordering and numeric representation may differ. The contract should adopt a deterministic canonical representation such as JCS or an equivalently precise project-defined projection [4].

7 Core execution model

7.1 State transition model

Let endpoint state at time t be represented as

$$S_t = \langle H, F, D, E, I, A, N, P, B, R \rangle, \tag{6}$$

where H is hardware/firmware, F storage/filesystems, D user/application data, E encryption state, I identity/authentication, A applications/dependencies, N network state, P policy/security controls, B boot configuration, and R recovery evidence/state.

A Core capability a with typed input x conceptually implements

$$F_a : (S_t, x, \alpha) \rightarrow (S_{t+1}, r, e), \tag{7}$$

where α is authority/policy context, r the typed result, and e durable evidence. This notation does not imply that every operation is transactional or reversible; it states that the semantics of the transition belong to Core.

7.2 Preconditions and postconditions

Every high-impact capability should define preconditions P_a and postconditions Q_a:

$$P_a(S_t, x, \alpha) = \text{true} \tag{8}$$

before mutation, followed by

$$Q_a(S_{t+1}) = \text{true} \tag{9}$$

for verified success. A zero process exit code without satisfied Q_a is not sufficient for a successful mutation result.

7.3 Stale-state and TOCTOU protection

Plans and approvals bind to observations that can become stale. Let $h_a(S)$ be a capability-specific fingerprint of safety-relevant state. Commit of a previously planned operation should require

$$h_a(S_{commit}) = h_a(S_{plan}) \tag{10}$$

or an explicit proof that changed fields are irrelevant to the capability. This prevents a stale Community window, delayed Fleet job, One-generated plan, or script from executing against changed storage, encryption, firmware, or target state.

7.4 Intent, artifact, target, allocation, and operation identities

High-impact migration work should not be represented by one overloaded identifier. The architecture separates distinct questions:

PlanId	Which reviewed migration intent and advisory state revision are being discussed?
ArtifactId	Which verified provider-scoped installer bytes are being referenced?
TargetId	Which stable physical target is being bound, independent of display order or disk renumbering?
AllocationId	Which exact future storage/layout mutation is intended? This is a target architecture concept and is not yet implemented at the implementation snapshot described below.
RecoveryId	Which recovery-readiness evidence is bound to the destructive operation, if sufficiently strong recovery evidence exists?
SecretHandle	Which locally protected sensitive input is available without placing secrets in argv, results, journals, or Fleet transport?
Approval	Which actor or policy authorizes the exact plan/target/allocation/artifact state and for what validity interval?
OperationId	Which durable execution instance owns lifecycle state across client-process exit and, where supported, reboot?

No identity above implicitly grants the authority of another. In particular, PlanId, ArtifactId, and TargetId are not execution permission. Destructive preparation should succeed only when the exact target and allocation can be revalidated, required artifacts remain verified, recovery evidence is sufficient for the risk class, secret custody is safe, and approval is bound to the same canonical intent.

7.5 Risk and authority

A practical architecture needs risk proportionality. An illustrative classification is:

Table 4: Illustrative capability risk classes.

Class	Example	Default control
R0	read-only inspection/discovery	automatic; logged as needed
R1	reversible local user-space change	explicit request; low blast radius
R2	privileged but bounded/recoverable change	policy + validated preconditions + operation journal
R3	storage/boot/destructive or difficult-to-reverse change	exact target binding, explicit authority, recovery evidence, exclusive locks, postcondition verification

The risk class is part of the capability contract and cannot be downgraded by a caller.

7.6 Privilege separation

CLI is the public process boundary into privileged Core operations, but that does not justify permanent elevation. Requests should be parsed and validated before privilege acquisition where feasible. Read-only capabilities should run without unnecessary elevation. Privileged execution may use scoped elevation or an internal broker/helper, but that internal mechanism remains behind Core and does not become a second public API.

This follows classical least-privilege and complete-mediation principles [5].

8 Durable operations and cross-reboot continuation

8.1 Core owns operation state

CLI is not the durable owner of an installation or migration. A migration may span Windows preparation, artifact staging, UEFI handoff, reboot, unattended Linux installation, first boot, and post-install verification. The invoking process cannot be assumed to survive.

Before irreversible work, Core MUST establish durable operation identity. The operation journal should bind at least:

- operation and request identifiers;
- capability/protocol versions;
- canonical input hash;
- relevant state revision;
- current lifecycle state;
- completed checkpoints;
- staged artifact identities/hashes;
- outstanding verification/recovery obligations;
- correlation identifiers for logs and evidence.

Sensitive credentials should not be duplicated into the journal merely for convenience.

8.2 Lifecycle

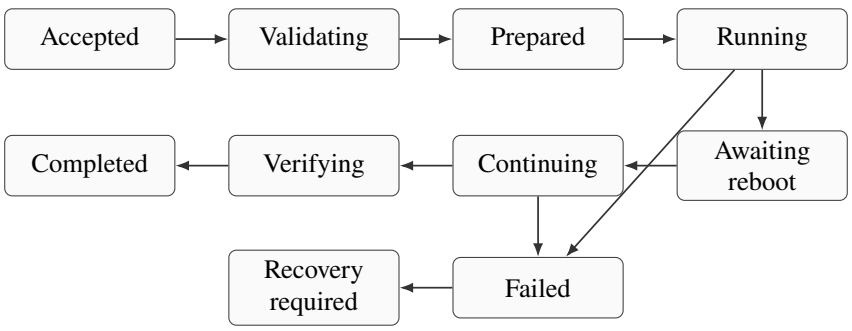


Figure 3: Generic durable operation lifecycle. Capability-specific state machines may add or omit states, but operation identity survives CLI process exit and reboot.

The existing migration engine already relies on UEFI one-shot boot selection and Windows/Linux handoff behavior; UEFI defines the relevant boot-manager variables and processing model [6]. The target architecture makes durable lifecycle state a Core concern visible through CLI rather than a GUI-owned workflow.

8.3 Progress and reconnection

For a running process, CLI may emit progress events. After CLI exit or reboot, a client reconnects using the durable operation ID through capabilities such as `operations.status`. A GUI closing must not corrupt an operation merely because the UI process ended.

8.4 Cancellation

Killing the CLI process is a transport failure, not the canonical cancellation mechanism. Cancellation must be explicit and capability-aware. Core may report states such as cancellable, cancellation-pending, temporarily non-cancellable, or recovery-required. For high-risk operations, "cancel" may mean "stop at the next safe boundary" rather than "restore the previous state."

8.5 Idempotency and retry

A client may retry after a timeout without knowing whether the original request reached Core. Therefore read-only capabilities should be idempotent, and mutating operations should bind durable request/operation identity strongly enough to detect replay. Core MUST NOT repeat an irreversible step solely because a client resubmitted an identical request.

9 Concurrency and resource ownership

Multiple clients can exist on one endpoint. Community may be open while Fleet has assigned work; a script may inspect state while One reasons about a plan. Safety therefore requires explicit resource ownership rather than UI convention.

Capabilities should declare resources they read or mutate, for example:

- `endpoint.state`;
- `storage.disk:<stable-id>`;
- `partition-table:<stable-id>`;
- `firmware.uefi`;
- `boot.configuration`;
- `migration.staging`;
- `recovery.state`;
- `iso-cache:<provider-id>`.

Read-only work may use shared leases where safe. Mutations require exclusive ownership for affected resources. Where separate processes can compete, purely in-process locks are insufficient; ownership must be enforceable across the relevant process boundary and durable where it spans reboot.

A lock conflict is a structured domain result such as `resource-busy`, not an incidental exception.

10 Secrets and sensitive data

10.1 No secrets in command-line arguments

Secrets MUST NOT be placed in argv. Command-line arguments may be observable through process inspection, logs, diagnostics, or crash reporting. Passwords, tokens, recovery keys, browser credentials, and similar material therefore require transient protected transport such as stdin/anonymous pipes initially, with room for a stronger local secret broker later.

The public protocol should prefer opaque secret references where possible. The Command Contract should distinguish a secret handle from the secret value itself.

10.2 Logging and results

Machine results, stderr diagnostics, operation journals, and Fleet evidence MUST NOT expose secrets by default. Public error semantics should be structured and sanitized. Debug logging can contain additional technical details only under explicit local policy and still requires redaction.

10.3 Fleet privacy boundary

Fleet transport models remain separate from local Command Contract models when their trust boundary differs. Fleet should upload allowlisted evidence rather than serialize a local migration manifest, user paths, Wi-Fi credentials, browser secrets, or arbitrary exception text. Sharing semantics does not require sharing every DTO.

11 Distribution-provider architecture

11.1 Provider role

The provider side isolates distribution-specific installer behavior from both clients and the generic migration engine. Existing iGloo concepts such as `IDistroPlugin` and a declarative installer boot specification illustrate the intended boundary.

A provider may supply:

- target metadata and requirements;
- provider-specific compatibility findings;
- installer configuration rendering;
- kernel/initrd/artifact and boot semantics;
- first-boot payloads;
- distribution-specific verification hooks;
- explicitly namespaced optional capabilities.

Linux installer automation differs materially across stacks such as Kickstart, Debian preseed, and Ubuntu autoinstall [7–9]. Core therefore normalizes lifecycle and safety while providers own the target-specific delta.

11.2 Provider constraints

Providers MUST NOT become alternate uncontrolled execution engines. General disk, partition, firmware, and operation-journal semantics remain Core responsibilities. Providers declare/render target behavior; Core controls when those artifacts are used and under which validated state.

A future open provider ecosystem will require publisher identity, signing, revocation, and potentially sandboxing. Provider binaries should be treated as trusted code until those mechanisms are defined.

12 Client integration

12.1 iGloo Community

Community should reach endpoint capabilities only through the CLI machine protocol. A graphical workflow may collect richer local presentation state, but operational behavior should be reproducible from a machine request. This yields a direct debugging benefit: a Community defect can be reduced to CLI request, protocol/version, endpoint evidence, Core result, and renderer behavior rather than requiring UI automation to reproduce execution semantics.

Closing Community should not implicitly cancel an operation. Reopening Community should be able to reconnect by operation ID when durable work is active.

12.2 iGloo One

One is a reasoning/conversational client, not an execution engine. It should derive or adapt tool definitions from capability discovery rather than maintain a parallel private set of system commands.

A simplified loop is

$$O_t \xrightarrow{R_\theta(I,O_t,K)} q_t \xrightarrow{\text{CLI}} r_t \to O_{t+1}, \tag{11}$$

where I is user intent, O_t current observations, K context/knowledge, R_θ a replaceable reasoning system, q_t a typed iGloo capability request, and r_t the actual structured result.

Tool-using language-model research provides useful background [10, 11], but iGloo narrows the action space deliberately because migration operations can be destructive. One follows these rules:

- no direct Core/Runtime/Preflight/Migration references;
- no arbitrary privileged shell as the normal tool interface;
- actual CLI results feed the next reasoning step;
- authority and approval remain outside the reasoning model;
- untrusted content is data, not policy;
- secrets should be referenced through opaque handles rather than inserted into model context;
- model/provider choice remains replaceable and does not change Core semantics.

12.3 iGloo Fleet

Fleet keeps a strict distinction between orchestration and execution.



Figure 4: Fleet execution boundary. The Server assigns and authorizes work; the endpoint Agent invokes the same local CLI used by other clients.

Fleet Server MUST NOT invoke CLI or load Core. Fleet Agent translates Fleet work into local Command Contract requests and translates local results into sanitized Fleet evidence. Remote destructive execution remains out of scope until recovery, authority, target binding, rollout containment, and evidence requirements are proven.

The current Fleet direction based on TLS/mTLS is compatible with established TLS and X.509 mechanisms [12, 13]; transport authentication does not replace application-level authorization.

12.4 Third-party and distro-owned frontends

A distribution maintainer may build a branded installer/migrator that never depends on Community:

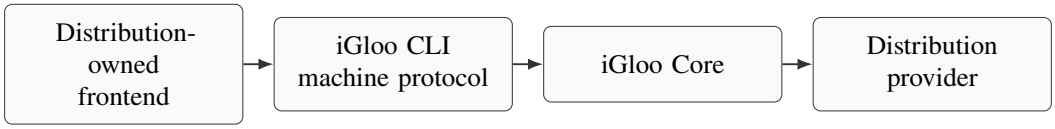


Figure 5: A distribution may own both its user experience and its provider while reusing the common engine and CLI contract.

The same model applies to OEMs, MSPs, public-sector custom frontends, test labs, and scripts.

13 Security and trust boundaries

13.1 Threat model

Clients can be buggy, stale, compromised, or semantically incorrect. This includes first-party clients. CLI therefore forms a protection boundary, and Core still performs independent validation.

Table 5: Selected threats and architectural responses.

Threat	Example	Architectural response
Malformed request	Client emits inconsistent parameters	Schema and cross-field validation before Core mutation
Stale state	Disk/encryption state changes after planning	Capability-specific state revision and revalidation
Reasoning error	One invents an unsupported action	Capability discovery + typed schemas; no arbitrary privileged shell
Prompt injection	Untrusted content asks One to bypass policy	Authority/policy outside reasoning context
Concurrent mutation	Fleet and Community both target storage	Cross-process resource ownership + structured conflict
Replay	Client retries after timeout	Durable operation/request identity and idempotency rules
CLI substitution	First-party app invokes wrong executable	Trusted deterministic CLI resolution + protocol/product validation
Secret leakage	Password placed in argv or logs	Protected transient input; redaction; secret handles
Provider compromise	Malicious provider attempts unsafe mutation	Provider trust policy; general mutation remains Core-owned
Fleet blast radius	Bad job targets many endpoints	Canaries, staged cohorts, concurrency limits, halt thresholds
Evidence tampering	Approval no longer matches plan	Canonical hashes and immutable correlation
Cross-reboot partial failure	Machine fails between Windows/Linux phases	Durable journal, one-shot boot semantics where applicable, recovery obligations

13.2 CLI location and substitution resistance

First-party applications SHOULD NOT search arbitrary `PATH` entries for a compatible CLI. Packaging should define a deterministic trusted location (for example, alongside the installed product or through trusted installation metadata), and the client should verify product/protocol compatibility before sending a request.

13.3 Approval artifacts

A general R2/R3 authorization model needs more than a boolean `-yes`. An approval artifact should bind to the exact capability, canonical input, relevant state/evidence revision, actor/policy decision, and validity interval. Community can obtain approval interactively; Fleet can bind it to audited operator approval; One can request approval but cannot manufacture it.

14 Recovery and failure containment

14.1 Recovery vocabulary

The architecture distinguishes:

- **reversible**: a tested compensation operation can restore the prior state;
- **recoverable**: exact prior state may not be restored, but a documented safe/bootable state can be reached;
- **irreversible**: information/state is destroyed without a verified restoration path.

Capabilities should declare which category applies and which recovery evidence is required before execution.

14.2 Verification

For an operation with intended postcondition Q,

$$\text{Success}(a) \Rightarrow Q(S_{t+1}) = \text{true}. \tag{12}$$

If the required postcondition cannot be established, the result should be `verification-failed` or a capability-specific incomplete state rather than success.

14.3 CLI process failure versus operation failure

A CLI transport can fail while a durable Core operation continues. The protocol must distinguish:

- CLI process could not start;
- protocol negotiation failed;
- transport broke after acceptance;
- Core rejected the request before execution;
- Core operation failed;
- Core operation requires recovery;
- operation continues and must be reconnected by ID.

Flattening all cases to process exit code 1 is insufficient for first-party or third-party automation.

15 Observability and evidence

Every operation should carry one correlation identity across client, CLI, Core, provider, Fleet Agent, and Linux-side continuation where applicable. Structured evidence may include:

- capability and versions;
- request/operation IDs;
- precondition observations and state revision;
- authority/policy reference;
- provider identity/version;
- artifact hashes and provenance;
- checkpoint transitions;

- postcondition observations;
- sanitized failure details;
- recovery actions and final state.

Evidence is not automatically telemetry. Local clients may retain evidence locally. Fleet uploads remain explicitly allowlisted and privacy-filtered.

16 Conformance and compatibility

16.1 Client conformance

A conformant machine client should demonstrate that it:

- resolves and verifies a compatible CLI;
- negotiates protocol/capability versions;
- sends schema-valid requests;
- handles event/result framing correctly;
- does not parse human output;
- preserves correlation and operation identity;
- distinguishes transport from domain failures;
- respects risk/authority semantics;
- does not put secrets in argv.

16.2 Provider conformance

A provider conformance suite should verify metadata/schema validity, compatibility evaluation, deterministic rendering where feasible, boot specification completeness, first-boot payload integrity, absence of client dependencies, and representative installer/hardware fixtures.

16.3 First-party semantic parity

The strongest regression barrier is cross-client equivalence. For every migrated capability, tests should prove that Community, One tool adapters, Fleet Agent translation, direct machine invocation, and direct human CLI eventually reach the same capability implementation. Result presentation may differ; execution semantics must not.

16.4 Architecture tests

The build should fail if the intended dependency graph regresses. At minimum:

- Community, One, and Fleet Agent may depend on CLI Client and public contract models but MUST NOT reference Core.Host or internal execution modules;
- CLI may reference Core.Host;
- Fleet Server MUST NOT reference CLI or Core;
- providers MUST NOT reference clients;
- CLI Client MUST NOT contain execution logic;
- machine-mode output purity should be tested as a protocol invariant.

17 Reference interaction flows

17.1 Community inspection

1. Community creates a typed `system.inspect` request.
2. CLI Client locates and verifies the installed CLI.
3. Request is written to machine-mode stdin.
4. CLI validates protocol/schema and invokes the Core capability.
5. Core produces typed observation/evidence.
6. CLI emits a machine result.
7. CLI Client deserializes it; Community renders graphical state.

At no stage does Community need to know whether the observation came from WMI, registry, firmware APIs, or another internal implementation.

17.2 One tool use

1. One discovers available capabilities and schemas.
2. The model/tool adapter selects `migration.plan` based on user intent and prior observations.
3. CLI performs normal validation and Core invocation.
4. One receives a structured plan, blockers, warnings, and uncertainty.
5. One explains the result or requests the next explicit capability.

No model-generated shell script is necessary.

17.3 Fleet dry-run

1. Fleet Server assigns a typed, authorized dry-run work item.
2. Fleet Agent validates ownership/expiry and translates it to local CLI request semantics.
3. CLI/Core perform local read-only assessment/planning.
4. Fleet Agent sanitizes the result according to Fleet's privacy schema.
5. Server stores evidence and applies Fleet-specific approval rules.

Fleet transport and Command Contract types may intentionally differ because their trust boundaries differ.

17.4 Distribution-owned frontend

1. A distribution ships/registers an iGloo-compatible provider.
2. Its independent frontend discovers provider support through CLI.
3. The frontend performs compatibility and planning through the machine protocol.
4. Core invokes the provider only for provider-owned target-specific behavior.
5. General storage, boot, journaling, and verification semantics remain common.

17.5 Cross-reboot migration

1. Client requests an authorized migration preparation capability.
2. Core validates state, acquires locks, creates durable operation ID, and stages verified artifacts.
3. CLI reports `AwaitingReboot`; the process may exit.
4. Core's existing handoff mechanism continues through the staged installer and first-boot environment.
5. After the system returns to a normal reachable state, a client queries `operations.status` or `migration.verify` by operation ID.
6. Core verifies declared postconditions and produces final evidence.

18 Implementation status and migration plan (non-normative)

18.1 Status at publication

The implementation snapshot reviewed for this version is the current working state of branch `refactor/community-fleet-foundation` on 6 October 2026. It is intentionally described as an implementation snapshot rather than as proof that the complete target architecture is finished.

The executable CLI boundary is real and validated. `Igloo.Cli.exe` and Community's `iGloo.exe` remain distinct applications. `Igloo.Cli.Client` resolves a known CLI location, validates the CLI/product/protocol hello before sending a request, and exchanges typed requests and JSON Lines events over redirected standard streams. Human and machine CLI modes reach the same dispatcher. One already uses the CLI Client for read-only/planning access.

The current public command surface includes system/version/capability discovery; system inspection and detailed inspection; distro discovery and compatibility; migration inspection and advisory planning; durable PlanId reopening; ISO/artifact acquisition and status; operation/status queries; recovery inspection; and `migration.target.bind`. Artifact acquisition uses the existing verified ISO path and records an opaque provider-scoped ArtifactId without exposing the private cache path. Migration Plan V2 persists non-secret advisory metadata and revision fingerprints. `migration.target.bind` reloads a saved plan, revalidates provider/machine observations, and binds the plan to an opaque TargetId whose private record stores canonical target identity rather than raw public device identifiers. Disk renumbering alone does not invalidate the binding; a changed or replaced target does.

These bindings are deliberately non-executable. TargetId currently identifies the Windows boot disk, not an exact future free-space allocation. `recovery.inspect` still reports unavailable exact readiness. There is no public SecretHandle, exact AllocationId, execution-bound approval artifact, migration OperationId producer, installer/first-boot continuation protocol, or postcondition verifier. Therefore `migration.prepare`, `migration.execute`, and `migration.verify` remain unregistered.

Community still uses private Core.Host workflows for its mutation-heavy path, including cleanup, ISO handoff, staging, USB, and installation preparation. Fleet Agent's trusted inspection/planning entry uses CLI, while Host-backed adapters remain for transition/fixture testing; Fleet Server remains isolated and no remote destructive work type exists. The current file operation journal and cross-process leases are proven for ISO acquisition and selected private host operations, but they do not yet constitute a resumable Windows-to-installer-to-first-boot migration lifecycle.

At this snapshot, full solution restore and build succeed with zero warnings/errors and 1,495 tests pass. Fleet Phase 0 and Phase 1 validation remain read-only/non-executable. These numbers are evidence for this implementation snapshot only, not a permanent property of the architecture.

18.2 Remaining convergence sequence

The remaining work should continue in safety order rather than dependency-graph order:

1. define an exact non-destructive storage allocation binding (separate from TargetId) so the intended source partition, geometry, shrink/free-space assumptions and resulting allocation can be revalidated;
2. audit sensitive-input lifetimes and establish local secret custody without exposing MigrationManifest or secrets through public protocol state;
3. strengthen recovery inspection only to the level actually proven by recovery evidence; do not fabricate readiness;
4. define client-neutral approval binding to the exact plan, target, allocation, artifacts and state revisions;
5. implement `migration.prepare` only when those prerequisites can be revalidated together, and only then create a durable migration OperationId;
6. extend operation journaling from ISO acquisition to migration lifecycle state and durable target ownership;
7. bind Windows preparation, installer handoff and Linux first boot to the same OperationId and add replay-safe continuation;
8. register `migration.execute` only for mutation subsets whose target, authority, recovery, replay and continuation gates are proven;
9. implement postcondition-driven `migration.verify`;
10. migrate Community's destructive workflow from Core.Host to CLI only after parity is demonstrated, then remove the remaining Host edge;
11. remove Fleet Agent transition Host adapters after equivalent CLI semantics cover their tested role; keep Fleet Server isolated;
12. publish external conformance commitments only after first-party clients have demonstrated the executable boundary end to end.

The migration strategy explicitly avoids a big-bang rewrite of already validated disk, boot, installer, and migration code.

19 Validation strategy

19.1 Protocol tests

For each capability version, tests should cover:

- valid and invalid schemas;
- unknown field/enum policy;
- protocol and capability-version negotiation;
- stdout protocol purity;
- JSON/event framing;
- stable error mapping;
- request/evidence canonicalization where used;
- timeouts and process-start failure;
- cancellation boundaries;
- stale-state conflicts;
- idempotent replay;

- resource conflicts;
- absence of secrets in argv/logs.

19.2 Core safety tests

High-impact operations require VM and physical-hardware tests with injected failures: network interruption, disk-full conditions, corrupt artifacts, firmware quirks, encryption state, Secure Boot combinations, duplicate/reordered devices, installer failure, power/reboot interruption, and recovery-path failure.

19.3 Cross-client parity

For every migrated first-party capability, test semantic equivalence across at least direct human CLI, direct machine CLI, Community through CLI Client, and Fleet Agent translation where applicable. One tool adapters should be tested for schema/tool mapping independently from model quality.

19.4 Provider matrix

Provider conformance must be validated across multiple installer stacks and representative hardware. The purpose of the CLI/Core boundary is to prevent the hard-won target-specific rules from leaking upward into each new client.

20 Open engineering questions

The architecture is intentionally explicit about questions that remain implementation decisions:

1. **Exact allocation identity.** TargetId identifies a stable physical target, but a destructive operation also needs a separately revalidatable binding to the exact source partition, geometry, shrink/free-space assumptions and future allocation.
2. **Secret custody.** The real Community staging flow must be audited to determine which sensitive values are needed only before staging, which must survive process exit, and whether any must cross the Windows-to-Linux boundary. Public command state must not contain MigrationManifest or plaintext secrets.
3. **Recovery readiness.** A captured observation is not automatically a proven recovery path. The exact evidence required before each R3 mutation must be defined from implemented recovery capabilities.
4. **Approval representation.** Local Community approval, direct technical CLI use, and future Fleet audited approval need one client-neutral authorization artifact bound to the exact canonical intent and validity interval.
5. **Durable operation ownership.** Process leases are insufficient across reboot. The journal must make it possible to determine that an active OperationId owns a target even when the initiating process no longer exists.
6. **Cross-reboot continuation.** Windows preparation, installer execution, Linux first boot and verification need replay-safe correlation to one OperationId without exposing private handoff state.
7. **Privilege broker.** Capability-scoped elevation is preferred, but the exact Windows helper/broker design requires implementation and threat analysis.
8. **Provider trust.** Signing, publisher identity, revocation, and sandboxing policy are required before treating arbitrary third-party providers as safe.
9. **Offline Fleet jobs.** Disconnected execution may require object-level work signatures and independent expiration in addition to mTLS.

10. **Specification governance.** If external vendors depend on the protocol, capability evolution needs a public deprecation/proposal process rather than repository-internal refactoring alone.

21 Conclusion

The final iGloo architecture is intentionally narrow at its public boundary.

iGloo Core is the execution engine. *iGloo CLI* is the single canonical executable gateway to that engine. *iGloo Community*, *iGloo One*, the Fleet endpoint Agent, humans, scripts, and independent third-party tooling all consume endpoint capabilities through the same CLI semantics. The *iGloo Command Contract* defines the machine-readable protocol exposed by CLI, while distribution providers extend target-specific installation behavior below Core.

The value of the design is not that every internal module disappears. The opposite is expected: Core can remain internally decomposed into inspection, ISO acquisition, migration, recovery, USB, runtime, provider, and platform-specific components. Sustainability comes from encapsulating that complexity behind one tested gateway. A new client no longer needs to learn the engine's assembly graph, and a new provider no longer requires changes in each frontend.

The architecture is successful when the same capability means the same thing regardless of who requested it; when Community can be replaced by a distro-owned frontend without rewriting migration logic; when One can reason over typed tools without receiving unrestricted system authority; when Fleet can orchestrate endpoints without turning the control plane into a remote execution engine; and when CLI process failure, Core operation failure, verification failure, and recovery-required states remain distinguishable.

In that model, iGloo is no longer architecturally defined by a particular GUI. It is defined by a reusable execution engine and a stable executable interface to that engine.

Implementation evidence boundary

This paper deliberately separates architecture from implementation maturity. Any claim that a capability, provider, recovery path, hardware configuration, or Fleet execution mode is production-ready should be supported by separate versioned test/deployment evidence. The presence of a capability in the target taxonomy is not itself an implementation claim.

Author

Gilles D'huyvetter: architecture, product conception, migration-engine design, implementation direction, technical specification and review.

Competing interests

The author is the founder of the iGloo project and may have a commercial interest in its development and adoption.

References

[1] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. Tech. rep. RFC 8259. IETF, 2017. DOI: 10.17487/RFC8259. URL: https://www.rfc-editor.org/rfc/rfc8259.html.

[2] Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. *JSON Schema: A Media Type for Describing JSON Documents, Draft 2020-12*. Tech. rep. JSON Schema Project, 2022. URL: https://json-schema.org/draft/2020-12/.

[3] Tom Preston-Werner. *Semantic Versioning 2.0.0*. Specification. 2013. URL: https://semver.org/spec/v2.0.0.html.

[4] Anders Rundgren, Bret Jordan, and Samuel Erdtman. *JSON Canonicalization Scheme (JCS)*. Tech. rep. RFC 8785. RFC Editor, 2020. DOI: 10.17487/RFC8785. URL: https://www.rfc-editor.org/rfc/rfc8785.html.

[5] Jerome H. Saltzer and Michael D. Schroeder. "The Protection of Information in Computer Systems". In: *Proceedings of the IEEE* 63.9 (1975), pp. 1278–1308. DOI: 10.1109/PROC.1975.9939. URL: https://web.mit.edu/saltzer/www/publications/protection/.

[6] UEFI Forum. *Unified Extensible Firmware Interface Specification, Release 2.11*. See Section 3, Boot Manager. 2024. URL: https://uefi.org/specs/UEFI/2.11/.

[7] Fedora Project. *Kickstart Documentation*. URL: https://docs.fedoraproject.org/en-US/fedora/latest/install-guide/advanced/Kickstart_Installations/.

[8] Debian Project. *Debian Installation Guide: Automating the installation using preseeding*. URL: https://www.debian.org/releases/stable/amd64/apb.en.html.

[9] Canonical. *Automated Server Install / Autoinstall Reference*. URL: https://canonical-subiquity.readthedocs-hosted.com/en/latest/reference/autoinstall-reference.html.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.

[11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. DOI: 10.52202/075280-2997. URL: https://papers.nips.cc/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[12] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. Tech. rep. RFC 8446. IETF, 2018. DOI: 10.17487/RFC8446. URL: https://www.rfc-editor.org/rfc/rfc8446.html.

[13] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and W. Timothy Polk. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. Tech. rep. RFC 5280. IETF, 2008. DOI: 10.17487/RFC5280. URL: https://www.rfc-editor.org/rfc/rfc5280.html.